

DP Computer Science: B 2.1.2 String Manipulation (Python)

1. Core Concepts & Learning Objectives

Learning Intentions

By the end of this lesson, students will be able to:

- Define the terms "string," "substring," and "concatenation"
- Explain that strings are 0-indexed and immutable
- Construct new strings by concatenating two or more strings using the + operator
- Extract a substring from a given string using slicing ([start:end])
- Alter a string by replacing a known substring using the .replace() method
- Manipulate a string by combining slicing ([:]) and concatenation (+) to build a new string

Key Vocabulary

Term	Definition
String	A sequence of characters (e.g., "Hello" , "Python")
Substring	A smaller sequence of characters extracted from a string
Concatenation	Joining two or more strings together using the + operator
0-indexed	The first character in a string is at index 0
Immutable	Strings cannot be changed in place; operations create new strings
Slicing	Extracting a portion of a string using [start:end] syntax
Replacement	Changing a substring using the .replace() method

Approaches to Learning (ATL)

Skill	Description
Thinking (Acquisition of Knowledge)	Learning and understanding the specific syntax for Python's string methods
Thinking (Application)	Applying knowledge of string methods and slicing to solve practical problems
Self-Management (Organisation)	Following step-by-step logic to construct manipulated strings

2. Introduction: What is a String?

The "String Surgery" Hook

| "How does a website create a username like 'Welcome, Bob!'? How does Word 'find and replace' work?"

What is a String?

A string is a sequence of characters. In Python, strings are enclosed in quotes (single or double).

0-Indexing

text

0-INDEXING IN STRINGS

String: H E L L O

Index: 0 1 2 3 4

H	E	L	L	O
---	---	---	---	---

0 1 2 3 4

✔ First character is at index 0

✔ Last character is at index len(string) - 1

len("Hello") = 5

Last index = 5 - 1 = 4

The Train Analogy

"A string is like a train. Each character is in a car, and the first car is car #0."

String Properties

Property	Description	Example
Sequence	A string is a sequence of characters	"Hello" is 5 characters
0-indexed	First character at index 0	"Hello"[0] = 'H'
Immutable	Cannot be changed in place	Must create a new string
Length	Use len() to find length	len("Hello") = 5

3. Core Concept: Concatenation (+)

What is Concatenation?

Concatenation is the process of joining two or more strings together using the + operator.

Examples

python

```
# Basic concatenation
first_name = "Bob"
last_name = "Smith"
full_name = first_name + " " + last_name
```

```
print(full_name) # Output: Bob Smith

# Concatenating multiple strings
greeting = "Hello"
name = "Alice"
message = greeting + ", " + name + "!"
print(message) # Output: Hello, Alice!

# Concatenating with variables and literals
age = 25
# String must be explicitly converted using str()
sentence = "I am " + str(age) + " years old."
print(sentence) # Output: I am 25 years old.
```

Key Points

Point	Description
Operator	<code>+</code> joins strings together
Spaces	Spaces are not added automatically; include them in quotes
Numbers	Numbers must be converted to strings using <code>str()</code>

4. Core Concept: Extraction (Slicing)

What is Slicing?

Slicing is the process of extracting a portion of a string using the `[start:end]` syntax.

Slicing Syntax

text

SLICING SYNTAX

string[start:end]

✔ start is inclusive

✘ end is exclusive (stops before end)

Example: "Hello"[0:3] = "Hel"

Index: 0 1 2 3 4

H E L L O

▲ ▲

start=0 end=3 (stops before index 3)

Result: H E L → "Hel"

Slicing Examples

```
python

greeting = "Hello World"

# Extract "Hello"
word1 = greeting[0:5]    # Index 0 to 4 (stops before 5)
print(word1)             # Output: Hello

# Extract "World"
word2 = greeting[6:11]   # Index 6 to 10 (stops before 11)
print(word2)             # Output: World

# Extract from start to a point
start = greeting[:5]     # Same as greeting[0:5]
print(start)             # Output: Hello

# Extract from a point to the end
end = greeting[6:]       # Same as greeting[6:11]
print(end)               # Output: World

# Extract with a step (optional)
steps = greeting[::2]     # Every second character
print(steps)             # Output: HLoWrld
```

Important Rule

Start is inclusive, end is exclusive.

Slice	Result	Explanation
"Hello"[0:2]	"He"	Characters at indices 0 and 1
"Hello"[1:4]	"ell"	Characters at indices 1, 2, and 3
"Hello"[:3]	"Hel"	From start to index 2
"Hello"[2:]	"llo"	From index 2 to the end
"Hello"[:]	"Hello"	The whole string

5. Core Concept: Altering and Replacing

Immutability

Strings are immutable—they cannot be changed in place. Any operation that modifies a string creates a new string.

```
python

name = "Alice"
name[0] = "X" # ❌ This will raise an error!

# Instead, create a new string
new_name = "X" + name[1:]
print(new_name) # Output: Xlice
```

Method 1: .replace()

The `.replace()` method replaces all occurrences of a substring with a new substring.

```
python

# Basic replacement
message = "I like cats"
new_message = message.replace("cats", "dogs")
print(new_message) # Output: I like dogs

# The original string is unchanged
print(message)     # Output: I like cats

# Replacing multiple occurrences
text = "one one one"
new_text = text.replace("one", "two")
print(new_text)    # Output: two two two
```

Method 2: Slicing + Concatenation

Combine slicing (`[ : ]`) and concatenation (`+`) to create a modified string.

```
python

# Changing a character
file = "report.txt"
# Change "txt" to "pdf"
new_file = file[0:7] + ".pdf" # Take first 7 chars + new extension
print(new_file) # Output: report.pdf

# Replacing a single character
name = "Hello"
new_name = "J" + name[1:] # Replace first character
print(new_name) # Output: JeLlO

# Inserting into a string
text = "Hello World"
# Insert " beautiful" after "Hello"
new_text = text[0:5] + " beautiful" + text[5:]
print(new_text) # Output: Hello beautiful World
```

`.replace()` vs. Slicing + Concatenation

Method	Use Case	Example
<code>.replace()</code>	Replace all occurrences of a substring	<code>"cats".replace("cats", "dogs")</code>
Slicing + <code>+</code>	Replace at a specific position	<code>"J" + name[1:]</code>

6. Guided Practice: Extracting the Domain from an Email

Problem

"Given an email `student@school.com`, how can we extract just the domain `school.com`?"

Steps

- Find the index of `'@'` using `.find()`
- Extract from the next character to the end using slicing

Code

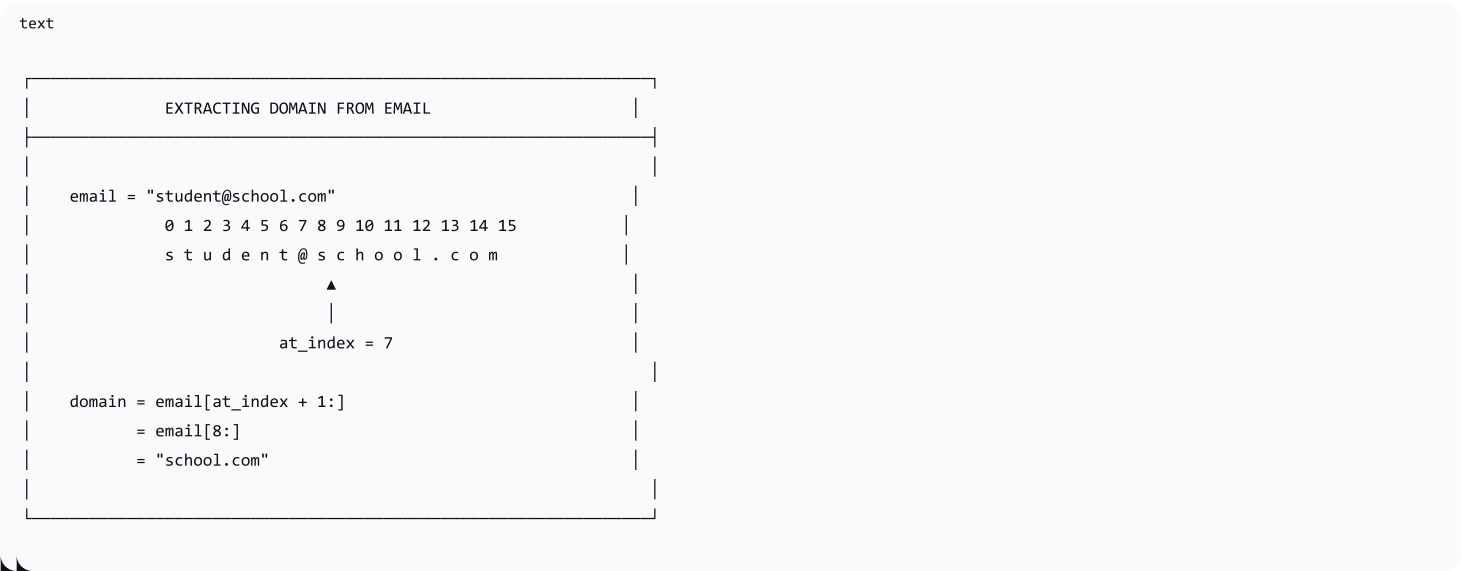
```
python

email = "student@school.com"

# Step 1: Find the position of '@'
at_index = email.find('@')
print(at_index) # Output: 7

# Step 2: Extract from after '@' to the end
domain = email[at_index + 1:] # Start at index 8
print(domain) # Output: school.com
```

Visual Representation



7. Independent Practice: Worksheet

Worksheet Questions

Part 1: Concatenation

- Write code to concatenate `firstName = "John"` and `lastName = "Doe"` with a space in between.
- Write code to create the message `"Hello, [name]!"` where `name` is a variable.

Part 2: Slicing

- Given `s = "Programming"`, extract the substring `"gram"`.
- Given `s = "abcdefghij"`, extract the substring from index 3 to index 7 (exclusive).

Part 3: Replacement

- Given `s = "I love apples"`, replace `"apples"` with `"oranges"`.
- Given `s = "Hello"`, change it to `"Jello"` using slicing and concatenation.

Part 4: Combined Manipulation

- Given `filename = "image.png"`, change the extension to `"jpg"`.
- Given `email = "user123@gmail.com"`, extract the username (part before `@`).

8. Lesson Sequence

Lesson Plan (60 minutes)

Phase	Time	Activity
Warm-Up & Introduction	5 min	"String Surgery" hook; review learning objectives
Core Concept 1: Strings	5 min	What is a string? 0-indexing; <code>len()</code>
Core Concept 2: Concatenation	7 min	<code>+</code> operator; examples
Core Concept 3: Slicing	13 min	<code>[start:end]</code> syntax; start inclusive, end exclusive
Core Concept 4: Replacing	10 min	<code>.replace()</code> ; slicing + concatenation
Guided Practice	5 min	Extract domain from email
Independent Practice	10 min	Students complete worksheet
Wrap-Up	5 min	Review; exit ticket

9. Assessment Activities

Assessment Type	Description
Class Participation	Observe engagement and contributions
Worksheet	Evaluate ability to use concatenation, slicing, and replacement
Exit Ticket	Gauge understanding and identify areas needing clarification

Sample Exit Ticket Question:

"Write the Python code to change the string `"Hello"` into `"Jello"`."

Expected Answer:

```
python
s = "Hello"
s2 = "J" + s[1:] # or s.replace("H", "J")
```

10. Differentiation

Level	Strategy
Support	Use the train analogy; provide a "cheat sheet" with syntax
Challenge	Explore advanced slicing (steps, negative indices); manipulate longer strings

11. Resources

- **Presentation** (Slide Deck)
- **eBook** (B2.1.2 with examples)
- **Worksheet:** Concatenation, slicing, replacement exercises
- **Worksheet Answers**

12. Summary: Key Takeaways

Operation	Syntax	Example	Result
Concatenation	<code>+</code>	<code>"Hello" + " " + "World"</code>	<code>"Hello World"</code>
Slicing	<code>[start:end]</code>	<code>"Hello"[0:2]</code>	<code>"He"</code>
Replacement	<code>.replace()</code>	<code>"cats".replace("cats", "dogs")</code>	<code>"dogs"</code>
Manual Replacement	<code>[:] + +</code>	<code>"J" + "Hello"[1:]</code>	<code>"Jello"</code>

text

KEY REMINDERS

Strings are sequences of characters

Strings are 0-indexed (first character is at 0)

Concatenation joins strings with +

Slicing extracts substrings with [start:end]

start is inclusive, end is exclusive

Strings are immutable (operations create new strings)

.replace() replaces substrings

"String manipulation is a fundamental skill in programming—concatenation, slicing, and replacement allow us to build and transform text data."